

Minería de Datos - Libro 1 - Clase 1 a 10

10. Minería de Texto - TensorFlow

Clase N° 10: Revisión de Temas Anteriores – Minería de Texto y Web con TensorFlow y Scikit-learn

Introducción

La Minería de Texto es una de las áreas más importantes dentro de la Minería de Datos, ya que permite analizar, procesar y extraer información valiosa a partir de datos no estructurados en formato textual. A diferencia de los datos numéricos o estructurados, el texto presenta desafíos como la ambigüedad del lenguaje, la variabilidad gramatical y la necesidad de preprocesamiento.

En esta clase abordaremos los conceptos clave y las herramientas necesarias para aplicar técnicas de Procesamiento de Lenguaje Natural (NLP) con TensorFlow y Scikit-learn, con el objetivo de convertir texto en información estructurada para su análisis y modelado predictivo.

Objetivos de la Clase

- Comprender la importancia y los fundamentos de la Minería de Texto.
- Aplicar técnicas de Procesamiento de Lenguaje Natural (NLP) para limpiar y analizar datos textuales.
- Utilizar modelos de Machine Learning para clasificación de texto.
- Explorar herramientas como Scikit-learn y TensorFlow en la construcción de modelos de minería de texto.

Importancia de la Minería de Texto en la actualidad

Las aplicaciones de la minería de texto han crecido exponencialmente con el auge de los datos digitales. Algunas de sus aplicaciones más comunes incluyen:

- **Análisis de Sentimientos:** Evaluar la percepción pública sobre productos, servicios o temas específicos en redes sociales.
- **Filtrado de Spam:** Detectar correos electrónicos no deseados mediante modelos de clasificación de texto.
- **Medicina:** Extraer información clave de registros médicos y diagnósticos automáticos.
- **Motores de Búsqueda:** Mejorar los resultados mediante análisis semántico y relevancia contextual.
- **Asistentes Virtuales:** Procesar texto y voz para interactuar con los usuarios (ej. Alexa, Google Assistant).

Ejemplo de Aplicación Real

Las empresas de comercio electrónico, como Amazon y MercadoLibre, utilizan Minería de Texto para analizar reseñas de productos y mejorar la experiencia del usuario. Mediante técnicas de análisis de sentimientos, pueden identificar productos con alta satisfacción o detectar problemas recurrentes para mejorar sus servicios.

Sección 1: Minería de Texto – Conceptos Fundamentales

¿Qué es la Minería de Texto?

La Minería de Texto es el proceso de descubrir patrones, tendencias o conocimientos a partir de datos textuales. Se basa en técnicas de Procesamiento de Lenguaje Natural (NLP) y Machine Learning para estructurar, analizar e interpretar grandes volúmenes de texto.

Flujo de Trabajo en Minería de Texto

1. **Preprocesamiento del Texto:**
 - Conversión a minúsculas.
 - Eliminación de signos de puntuación.
 - Tokenización (separación en palabras o frases).
 - Eliminación de palabras vacías (stopwords).
 - Lematización o Stemming (reducción de palabras a su raíz).
2. **Extracción de Características:**
 - Representación en Bolsa de Palabras (Bag of Words).
 - TF-IDF (Term Frequency - Inverse Document Frequency).
 - Embeddings de Palabras (Word2Vec, FastText, BERT).
3. **Análisis y Modelado:**
 - Clasificación de Texto (Ejemplo: Spam vs. No Spam).
 - Análisis de Sentimientos (Ejemplo: Opinión positiva vs. negativa).
 - Agrupamiento de Documentos (Ejemplo: Clustering de noticias similares).

Actividad Guiada de cada paso en Python

```
import nltk
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Descargar recursos necesarios
nltk.download("punkt")
nltk.download("stopwords")

# Texto de ejemplo
texto = "¡La minería de texto es fascinante! Permite analizar grandes volúmenes de datos."

# Conversión a minúsculas y eliminación de puntuación
texto = texto.lower().translate(str.maketrans("", "", string.punctuation))

# Tokenización
tokens = word_tokenize(texto)

# Eliminación de palabras vacías
tokens_limpios = [word for word in tokens if word not in stopwords.words("spanish")]
print("Texto Procesado:", tokens_limpios)
```

Explicación del Código:

- 1. Se convierte el texto a minúsculas para evitar inconsistencias.
- 2. Se eliminan signos de puntuación, ya que no aportan significado semántico.
- 3. Se realiza la tokenización, dividiendo el texto en palabras individuales.
- 4. Se eliminan las palabras vacías (stopwords), como "de", "es", "la", etc.

Ejemplo de Salida:

```
Texto Procesado: ['minería', 'texto', 'fascinante', 'permite', 'analizar', 'grandes', 'volúmenes', 'datos']
```

Conclusión: Después del preprocesamiento, el texto está limpio y listo para ser utilizado en modelos de Machine Learning.

1.1. Representaciones de Texto en Machine Learning

Para utilizar texto en modelos de Machine Learning, es necesario convertirlo en vectores numéricos.

1. Bolsa de Palabras (Bag of Words – BoW)

- Se crea una matriz de términos donde cada palabra representa una columna.
- Se cuenta la frecuencia de aparición de cada palabra en el documento.
- No considera el significado ni el orden de las palabras.

Ejemplo de Matriz de BoW

Documento	minería	texto	datos	permite
Doc 1	1	1	1	1
Doc 2	0	1	2	0

2. TF-IDF (Frecuencia de Término – Frecuencia Inversa de Documento)

- Penaliza palabras frecuentes que aparecen en muchos documentos.
- Da más peso a términos que son específicos de un documento en particular.

Actividad de Ejemplo en Python: Vectorización con TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Documentos de ejemplo
documentos = ["La minería de datos es importante",
              "El análisis de datos es clave en la IA"]

# Aplicar TF-IDF
vectorizador = TfidfVectorizer()
tfidf_matriz = vectorizador.fit_transform(documentos)

# Mostrar la matriz
print("Términos:", vectorizador.get_feature_names_out())
print("Matriz TF-IDF:\n", tfidf_matriz.toarray())
```

Ejemplo de Salida:

```
Términos: ['análisis', 'clave', 'datos', 'importante', 'ia', 'minería']
Matriz TF-IDF:
[[0.          0.          0.534522  0.534522  0.          0.534522]
 [0.534522  0.534522  0.267261  0.          0.534522  0.          ]]
```

Conclusión: TF-IDF mejora la representación de texto, dando más importancia a términos relevantes y menos peso a palabras comunes.

Sección 2: Introducción a TensorFlow y Scikit-learn para Minería de Texto

Objetivo de la Sección: En esta sección, exploraremos las principales herramientas para aplicar modelos de Machine Learning en Minería de Texto. En particular, veremos cómo utilizar TensorFlow y Scikit-learn, dos de las bibliotecas más utilizadas para el desarrollo de modelos de aprendizaje automático en Python.

2.1. TensorFlow: Deep Learning para Minería de Texto

¿Qué es TensorFlow?

TensorFlow es una biblioteca de código abierto desarrollada por Google para la implementación de modelos de Deep Learning. Se usa ampliamente en tareas de Procesamiento de Lenguaje Natural (NLP), incluyendo:

- Clasificación de texto (Ejemplo: detección de spam).
- Modelado de lenguaje (Ejemplo: autocompletar oraciones).
- Análisis de sentimientos (Ejemplo: opiniones positivas o negativas).
- Generación de texto (Ejemplo: traducción automática).

Arquitectura de TensorFlow para NLP

El flujo de trabajo típico en TensorFlow para minería de texto incluye:

1. **Preprocesamiento del texto:** Tokenización, eliminación de stopwords y vectorización.
2. **Creación del modelo:** Uso de redes neuronales recurrentes (RNN), redes convolucionales (CNN) o transformers.
3. **Entrenamiento:** Optimización con algoritmos como Adam o RMSprop.
4. **Evaluación y despliegue:** Prueba del modelo con nuevos datos.

Ejemplo en TensorFlow: Creación de una Red Neuronal para Clasificación de Texto

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Embedding, Dense, LSTM
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np

# Datos de entrenamiento
textos = ["Me encanta este producto", "Es un desastre, muy malo",
          "Excelente calidad y buen precio", "No me gustó, fue horrible"]
etiquetas = np.array([1, 0, 1, 0]) # 1 = Positivo, 0 = Negativo

# Tokenización del texto
tokenizer = Tokenizer(num_words=1000, oov_token="<OOV>")
tokenizer.fit_on_texts(textos)
sequences = tokenizer.texts_to_sequences(textos)

# Padding para que todas las secuencias tengan el mismo tamaño
padded_sequences = pad_sequences(sequences, maxlen=5, padding='post')

# Definir el modelo de red neuronal
modelo = keras.Sequential([
    Embedding(input_dim=1000, output_dim=16, input_length=5),
    LSTM(16),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compilar y entrenar el modelo
modelo.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
modelo.fit(padded_sequences, etiquetas, epochs=5, verbose=1)
```

Explicación del Código:

1. Se tokenizan los textos, convirtiéndolos en secuencias numéricas.

2. Se aplica Padding, asegurando que todas las oraciones tengan la misma longitud.
3. Se define el modelo con una capa Embedding, seguida de una capa LSTM (Long Short-Term Memory).
4. Se entrena la red neuronal para clasificación binaria (positivo/negativo).

Conclusión:

- Las redes neuronales permiten capturar relaciones complejas en texto.
- TensorFlow es ideal para tareas avanzadas como generación de texto y traducción automática.

2.2. Scikit-learn: Modelos Clásicos para Minería de Texto

¿Qué es Scikit-learn?

Scikit-learn es una biblioteca de Machine Learning diseñada para modelos tradicionales, como regresión, clasificación y clustering. A diferencia de TensorFlow, que es más adecuado para modelos complejos, Scikit-learn es ideal para tareas de minería de texto con algoritmos clásicos.

Ejemplo de Aplicaciones en Minería de Texto:

- Clasificación de documentos (Ejemplo: correos electrónicos spam vs. no spam).
- Análisis de sentimientos (Ejemplo: comentarios positivos o negativos).
- Agrupamiento de noticias (Ejemplo: clústeres de artículos similares).

Ejemplo en Scikit-learn: Clasificación de Opiniones con Naive Bayes

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

# Datos de entrenamiento
opiniones = ["Me encanta este producto", "Horrible, nunca lo compraría",
             "Excelente calidad y buen precio", "No me gustó, muy malo"]
etiquetas = [1, 0, 1, 0] # 1 = Positivo, 0 = Negativo

# Creación del modelo con TF-IDF y Naive Bayes
modelo_texto = make_pipeline(TfidfVectorizer(), MultinomialNB())
modelo_texto.fit(opiniones, etiquetas)

# Evaluación del modelo
nueva_opinion = ["Es un producto excelente y muy recomendable"]
print("Predicción:", modelo_texto.predict(nueva_opinion))
```

Explicación del Código:

1. Se usa TF-IDF para convertir el texto en una representación numérica.
2. Se entrena un modelo de Naive Bayes para clasificación.
3. Se realiza una predicción sobre una nueva opinión.

Ejemplo de Salida:

```
Predicción: [1] # El modelo clasifica la opinión como positiva.
```

Conclusión: Scikit-learn es más rápido y eficiente para modelos tradicionales de clasificación de texto. Los modelos como Naive Bayes o SVM son altamente interpretables y fáciles de entrenar.

2.3. Comparación entre TensorFlow y Scikit-learn

Característica	TensorFlow	Scikit-learn
Modelos soportados	Redes neuronales profundas	Algoritmos clásicos (SVM, árboles, regresión)
Facilidad de uso	Curva de aprendizaje más alta	API simple y rápida
Escalabilidad	Soporta GPU y TPU para modelos grandes	Limitado a CPU y memoria local
Preprocesamiento	TensorFlow Data API	Pandas y sklearn.preprocessing
Ejemplo de aplicación	Chatbots, traducción automática	Clasificación de texto, análisis de sentimientos

Conclusión General:

- Scikit-learn es una opción rápida y eficiente para modelos tradicionales de minería de texto.
- TensorFlow es más potente para modelos avanzados como redes neuronales recurrentes (RNN) o transformers.

- La elección entre TensorFlow y Scikit-learn dependerá del problema a resolver y la complejidad del modelo requerido.

Sección 3: Ejercicio Integrador y Publicación en el Foro

Objetivo de la Sección: En este ejercicio integrador, aplicaremos todo lo aprendido sobre Minería de Texto, desde el preprocesamiento de datos hasta la implementación de modelos de Machine Learning con Scikit-learn y TensorFlow. Finalmente, compartiremos los resultados en el foro, fomentando la discusión y el aprendizaje colaborativo.

3.1. Ejercicio Integrador: Análisis de Opiniones en Redes Sociales

Contexto: Una empresa quiere analizar opiniones de clientes en redes sociales para determinar si son positivas o negativas. Para ello, necesita un modelo de Machine Learning que clasifique automáticamente los comentarios en dos categorías: **1 (Positivo)** / **0 (Negativo)**.

Flujo de Trabajo del Ejercicio:

1. Preprocesar un conjunto de opiniones en español.
2. Aplicar TF-IDF para convertir el texto en características numéricas.
3. Entrenar un modelo de clasificación con Scikit-learn.
4. Comparar resultados con una red neuronal en TensorFlow.

Paso 1: Preparación de los Datos

```
import pandas as pd
import numpy as np
import nltk
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Descargar recursos necesarios de NLTK
nltk.download("punkt")
nltk.download("stopwords")

# Datos de ejemplo (opiniones de clientes)
opiniones = [
    "Este producto es excelente, me encantó",
    "Horrible, nunca lo compraría",
    "Muy bueno, lo recomiendo totalmente",
    "No me gustó, la calidad es mala",
    "Una compra increíble, vale la pena",
    "Muy malo, pésima experiencia",
]

# Etiquetas: 1 = Positivo, 0 = Negativo
etiquetas = np.array([1, 0, 1, 0, 1, 0])

# Función de preprocesamiento del texto
def limpiar_texto(texto):
    texto = texto.lower().translate(str.maketrans("", "", string.punctuation))
    tokens = word_tokenize(texto)
    tokens_limpios = [word for word in tokens if word not in stopwords.words("spanish")]
    return " ".join(tokens_limpios)

# Aplicar preprocesamiento
opiniones_limpias = [limpiar_texto(op) for op in opiniones]
```

Explicación del Código:

- Se convierten las opiniones a minúsculas y se eliminan signos de puntuación.
- Se realiza tokenización (separación en palabras).
- Se eliminan palabras vacías (stopwords).

Paso 2: Clasificación con Scikit-learn (Naive Bayes)

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Dividir en conjuntos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(opiniones_limpias, etiquetas,
                                                    test_size=0.2, random_state=42)

# Vectorización TF-IDF + Naive Bayes
modelo_nb = make_pipeline(TfidfVectorizer(), MultinomialNB())
modelo_nb.fit(X_train, y_train)

# Evaluar modelo
y_pred_nb = modelo_nb.predict(X_test)
print("Precisión Naive Bayes:", accuracy_score(y_test, y_pred_nb))
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred_nb))

```

Explicación del Código:

- TF-IDF convierte el texto en una representación numérica.
- Naive Bayes es un modelo eficiente para clasificación de texto.
- Se evalúa la precisión del modelo con un conjunto de prueba.

Paso 3: Clasificación con TensorFlow (Red Neuronal)

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Embedding, Dense, LSTM
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Tokenización del texto con TensorFlow
tokenizer = Tokenizer(num_words=1000, oov_token="<OOV>")
tokenizer.fit_on_texts(opiniones_limpias)
sequences = tokenizer.texts_to_sequences(opiniones_limpias)
padded_sequences = pad_sequences(sequences, maxlen=10, padding='post')

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(padded_sequences, etiquetas,
                                                    test_size=0.2, random_state=42)

# Definir el modelo de red neuronal
modelo_tf = keras.Sequential([
    Embedding(input_dim=1000, output_dim=16, input_length=10),
    LSTM(16),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compilar y entrenar el modelo
modelo_tf.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
modelo_tf.fit(X_train, y_train, epochs=5, verbose=1)

# Evaluar modelo
y_pred_tf = (modelo_tf.predict(X_test) > 0.5).astype("int32")
print("Precisión Red Neuronal:", accuracy_score(y_test, y_pred_tf))

```

Explicación del Código:

- Tokenización y padding con TensorFlow para convertir texto en secuencias numéricas.
- Red LSTM para aprender patrones en secuencias de texto.
- Evaluación del modelo con precisión.

Instrucciones para compartir resultados en el Foro

Publica un mensaje en el foro de la clase con los siguientes puntos:

- Resumen de los modelos aplicados y los resultados obtenidos.
- Capturas o gráficos de la precisión de Naive Bayes y la Red Neuronal.
- Comparación entre ambos modelos (ventajas y desventajas).
- ¿Cuál crees que es mejor para este problema? ¿Por qué?

Ejemplo de Publicación en el Foro

Análisis de Opiniones en Redes Sociales

Resumen del Análisis: "Entrené dos modelos para clasificar opiniones de clientes: un modelo de Naive Bayes con Scikit-learn y una Red Neuronal con TensorFlow. La precisión obtenida fue la siguiente:"

- Naive Bayes: 78% de precisión
- Red Neuronal (LSTM): 85% de precisión

Comparación entre modelos: "La red neuronal tiene mejor desempeño, pero requiere más datos y mayor tiempo de entrenamiento. En cambio, Naive Bayes es más rápido y fácil de interpretar."

Discusión en el foro:

- ¿Crees que una red neuronal siempre será mejor que Naive Bayes?
- ¿Cómo podríamos mejorar estos modelos con más datos?
- ¿Qué otras técnicas podríamos probar?

Bibliografía

1. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
2. Jurafsky, D., & Martin, J. H. (2021). *Speech and Language Processing (3rd Edition)*. Pearson.
3. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques*. Elsevier.
4. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*. O'Reilly.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
6. TensorFlow Developers. (n.d.). *TensorFlow: An Open Source Machine Learning Framework*. Recuperado de <https://www.tensorflow.org/>
7. Scikit-learn Developers. (n.d.). *Scikit-learn: Machine Learning in Python*. Recuperado de <https://scikit-learn.org/>
8. Hugging Face Developers. (n.d.). *Transformers Library*. Recuperado de <https://huggingface.co/transformers/>
9. Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *SciPy Conference*.